

# EVENT LOOP in 2 minutes

HA



# SINGLE-THREADED EXECUTION

Javascript operates on a single thread, meaning it executes one task at a time using the call stack, where functions are processed sequentially.

HA



# CALL STACK

Picture the call stack as a stack of plates. Each time a function is invoked, a new plate (function) is added to the stack. Once a function completes, the plate is removed.

HA



# WEB APIS

Asynchronous tasks like `setTimeout`, DOM events, and HTTP requests are managed by the browser's Web APIs, operating outside the call stack.

HA



# CALLBACK QUEUE

After an asynchronous task finishes, its callback is placed in the callback queue, which waits for the call stack to clear before moving forward.

HA



# EVENT LOOP

The event loop constantly monitors the call stack. When it's empty, the loop pushes the next callback from the queue onto the stack.

HA



# MICROTASKS QUEUE

Tasks like promises are placed in a microtasks queue, which has higher priority than the callback queue. The event loop checks the microtasks queue first to ensure critical tasks are handled immediately.

HA





# PRIORITY HANDLING

To sum up, the event loop prioritizes microtasks before handling other callbacks, ensuring efficient execution.

HA







# THANK YOU!

Keep learning  
Keep sharing

HA

